

DELUXE EPUB-TO-PDF PIPELINE ANALYSIS

Comprehensive Script Summary & Enhancement Roadmap

June 2026

Generated from deluxe_epub_to_pdf.py (~3,620 lines)

Project: local_deluxe_epub_pipeline_configurable_with_guide

CONTENTS

1. EXECUTIVE OVERVIEW	4
1.1 Purpose & Scope	4
1.2 Key Statistics	4
2. DETAILED ARCHITECTURE.....	6
2.1 Pipeline Flow	6
2.2 Regex Classification Engine	8
2.3 Data Models	9
2.4 EPUB Scanning & Classification	9
2.5 HTML Cleanup Pipeline	10
2.6 CSS Generation Engine	12
2.7 PDF Rendering & Optimization	12
2.8 QA Preflight System	13
3. AI INTEGRATION.....	15
3.1 Providers & Models	15
3.2 Structure Planning.....	15
3.3 Image Classification.....	15
3.4 Visual & Text QA.....	16
4. CONFIGURATION & AUTO-FIX SYSTEM	17
4.1 Three-Layer Override Precedence	17
4.2 Rule Pack System.....	17
4.3 Auto-Fix Engine.....	17
5. ENHANCEMENT ROADMAP	19
5.1 Architectural Improvements.....	19
5.2 Missing Typographic Features	20
5.3 Print-Production Additions	21
5.4 QA & Safety Enhancements.....	21
5.5 Workflow & UX.....	22

5.6 Output Diversification 22

5.7 Non-Latin & Accessibility 23

6. CONCLUSION & PRIORITY MATRIX24

1. EXECUTIVE OVERVIEW

1.1 Purpose & Scope

`deluxe_epub_to_pdf.py` is a local, single-script production pipeline that converts EPUB files into print-oriented A4 PDF interiors suitable for deluxe physical book editions. It targets large public-domain collected works (e.g., Delphi Classics, Project Gutenberg) and automates a strict professional book-production workflow.

The pipeline enforces: A4 trim, single-page PDF output, recto/verso start discipline, roman/Arabic pagination, running heads with collection/work title alternation, poetry/drama/cast-list normalization, functional vs. decorative image classification, promotional content removal, TOC page-number resolution, and rigorous PDF preflight QA with auto-fix capabilities.

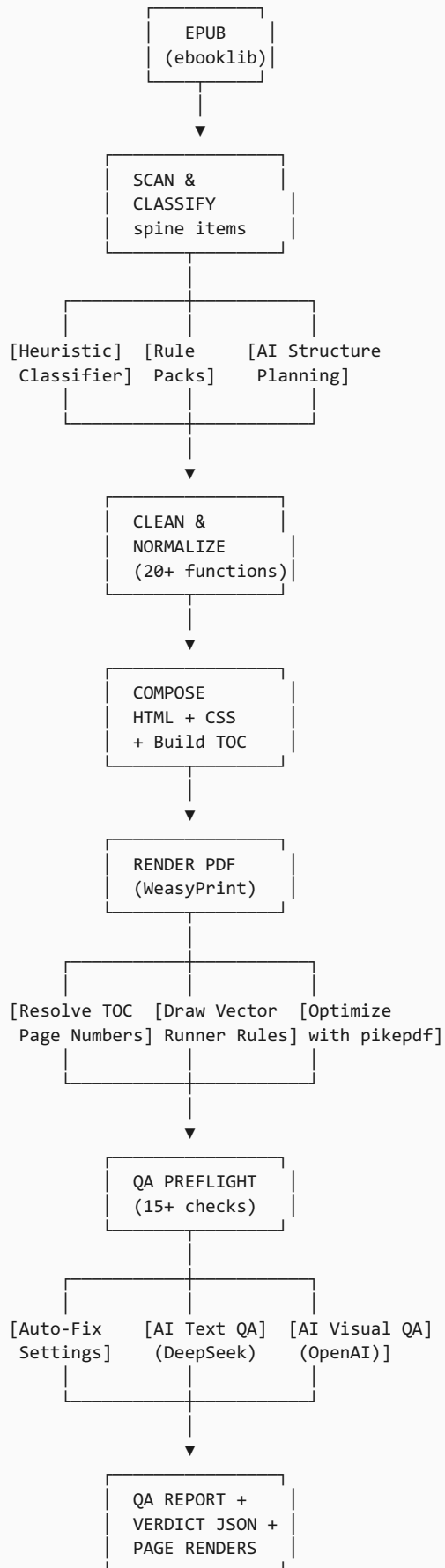
1.2 Key Statistics

3,620 LINES OF PYTHON	13 COMPILED REGEX PATTERNS
5 DATAClass MODELS	90+ CONFIGURABLE SETTINGS
20+ HTML CLEANUP FUNCTIONS	9 EXTERNAL DEPENDENCIES

Dependency	Role
BeautifulSoup4 + lxml	HTML parsing, cleanup, normalization
EbookLib	EPUB reading, spine extraction, metadata
WeasyPrint	HTML+CSS → PDF rendering engine
PyMuPDF (fitz)	PDF analysis, vector drawing, text extraction, page renders
pikepdf	PDF optimization, compression, linearization
openai	OpenAI/DeepSeek API for AI-assisted decisions
PyYAML	Config and rule-pack file parsing
Pillow	PNG render analysis, dark-page detection

2. DETAILED ARCHITECTURE

2.1 Pipeline Flow



2.2 Regex Classification Engine

The script defines **13 compiled regular expressions** as module-level constants, each serving a specific classification purpose:

Pattern	Purpose	Examples
PROMO_PATTERNS	Detect promotional/catalogue content	"Delphi Classics", "Also available", "Subscribe", "ISBN", "www.", "Goodreads"
BACKMATTER_PATTERNS	Identify back-of-book sections	"Notes", "Appendix", "Bibliography", "Glossary", "Index", "Chronology"
LOCAL_TOC_HEADINGS	Find inline mini-tables-of-contents	"Contents", "Table of Contents", "Chapter List"
COLLECTION_DIVISIONS	Detect collected-works divisions	"The Novels", "Short Stories", "Plays", "Poetry", "Essays"
MAJOR_WORK_HINTS	Identify major work boundaries	"Book I", "Part II", "Novel", "Play", "Poem"
CHAPTER_HEADINGS	Match chapter/section labels	"CHAPTER", "PART", "SCENE", "ACT", "Canto", Roman numerals
CAST_HEADINGS	Detect dramatis personae sections	"Dramatis Personae", "Characters", "Persons of the Play"
ACT_SCENE_HEADINGS	Identify act/scene transitions	"ACT I", "SCENE II"
PLATE_CAPTION_PATTERNS	Spot publisher-added plate captions	"Portrait of the Author", "Birthplace", "First Edition title page"
FUNCTIONAL_IMAGE_CLUES	Detect authorial/functional images	"Map", "Diagram", "Chart", "Rune", "Inscription", "Facsimile"
FUNCTIONAL_IMAGE_SRC_CLUES	Same as above for src attributes	Image filename contains "map", "diagram", "chart"
PUBLISHER_IMAGE_SRC_CLUES	Identify publisher/decorative images	"Cover", "Frontispiece", "Portrait", "Photo", "Logo", "Catalogue"
POETRY_CLASS_RE	Detect poetry-marked CSS classes	"poem", "poetry", "stanza", "verse", "canto", "song", "ode"

These patterns can be **extended at runtime** via YAML rule packs in the `rules/` directory. The `apply_rule_packs()` function merges external patterns into the built-in regexes using alternation. AI-generated suggestions are written as `.review.yaml` files for human approval.

2.3 Data Models

Five `@dataclass` classes organize all pipeline state:

Settings (~90 fields)

The complete typographic configuration — trim size, body font metrics, margin geometry, running head layout (6 variants), folio sizing, paragraph indent, blockquote margins, heading hierarchy (8 heading levels each with independent sizes and margins), TOC styling (4 levels), poetry verse block sizing, cast list layout, image policy (functional / keep-all / remove-all), front matter generation flags, rule pack paths, and pipeline behavior flags.

SpineDoc

Represents one EPUB spine item: index, item_id, name, href, raw bytes, extracted headings (up to 40), text sample (first 9,000 chars), text length, classification kind (one of 10 types), remove flag, major_title, current_division, poetry/drama/image boolean flags, and an AI confidence score.

TocEntry

Table of contents entry: hierarchical level (1–4), display title, target HTML anchor ID, and semantic kind (division / work / backmatter).

BuildLog

Complete audit trail: removed blocks, removed documents, kept/removed images, warnings, hard failures, AI decisions, title source, and counts for poetry blocks detected, cast sections detected, typographic fixes applied, and CSS auto-fix changes.

QAVerdict

Comprehensive quality-assurance state: page count, non-A4 pages, line spills, dark pages, blank page artifacts, header collisions, narrow columns, TOC warnings (page-number + duplicate), empty content pages, duplicate title pages, opener page warnings, work-description style warnings, first body folio warnings, OpenAI visual flags, text QA flags, AI rule suggestion file path, fonts seen, images seen, QA render paths. The `has_blockers` property aggregates all blocking conditions for `--strict` mode.

2.4 EPUB Scanning & Classification

The scanning phase reads EPUB spine items via EbookLib, then classifies each document through a two-tier system:

Tier 1: Heuristic Classification

`heuristic_classify_doc()` assigns each document a `kind` using the regex patterns. The priority chain:

1. **Promo detection** — Gutenberg license text or short documents matching `PROMO_PATTERNS`
2. **Local TOC detection** — "Contents" headings with low word count
3. **Backmatter** — matching `BACKMATTER_PATTERNS`
4. **Collection divisions** — matching `COLLECTION_DIVISIONS`
5. **Cast/drama** — matching `CAST_HEADINGS` or flagged as containing drama
6. **Poetry** — flagged by probe as poetry-containing
7. **Major works** — H1 headings that aren't chapter-like, less than 100 chars
8. **Frontmatter** — preface/introduction/foreword near the start
9. **Fallback: chapter** — everything else

Tier 2: AI-Assisted Classification (optional)

`apply_openai_book_plan()` sends spine documents to OpenAI or DeepSeek in batches of 24, each with headings, text samples (3,500 chars), text length, and local heuristic flags. The AI returns structured JSON with `kind`, `remove_document`, `major_title`, `current_division`, and confidence scores. AI decisions only override when confidence ≥ 0.86 for removals.

2.5 HTML Cleanup Pipeline

Each non-removed document passes through ~20 sequential normalization functions inside `clean_document()`:

#	Function	What It Does
1	<code>remove_comments_scripts_styles</code>	Strips HTML comments, <code><script></code> , <code><style></code> , <code><iframe></code> , <code><object></code> , <code><embed></code>
2	<code>strip_bad_attributes</code>	Removes all attributes except href, src, alt, title, id, class, colspan, rowspan
3	<code>remove_local_mini_tocs</code>	Detects "Contents" headings followed by link-heavy nav blocks; strips them
4	<code>remove_compact_local_contents_blocks</code>	Catches link-dense nav blocks without explicit headings
5	<code>remove_promotional_blocks</code>	Strips <code><section>/<div>/<p></code> matching promo patterns with structural clues
6	<code>rewrite_images</code>	For each <code></code> , classifies keep/remove; preserves functional images, removes plates
7	<code>remove_empty_layout_shells</code>	Strips empty <code><figure>/<div>/<section></code>
8	<code>unwrap_useless_inline_tags</code>	Unwraps empty <code>/</code> tags
9	<code>simple_typographic_cleanup</code>	Converts double-dashes to em-dashes, triple-dots to ellipsis, removes double spaces
10	<code>normalize_notes_refs</code>	Converts footnote reference links into <code><sup class="note-ref"></code>
11	<code>normalize_poetry</code>	Converts <code>
</code> -lineated blocks to structured verse blocks; groups short-paragraph runs
12	<code>normalize_cast_and_drama</code>	Marks cast headings, wraps cast lists, detects stage directions and act/scene transitions
13	<code>promote_paragraph_headings</code>	Detects disguised headings in <code><p>/<div></code> and promotes them to <code><h2></code>
14	<code>add_synthetic_opener_if_needed</code>	Inserts synthetic <code><h1></code> when AI-assigned <code>major_title</code> doesn't appear in the document
15	<code>normalize_headings</code>	Core heading classifier — determines heading semantic role, builds TOC entries
16	<code>mark_major_work_descriptions</code>	Detects editorial blurbs after work titles; styles as smaller italic apparatus
17	<code>insert_major_opener_separators</code>	Inserts true-blank divs before/after major opener pages
18	<code>remove_duplicate_current_work_title_line</code>	

		Removes repeated work title lines in continuation documents
19	<code>mark_standalone_work_description_fragment</code>	Catches editorial blurbs split into separate EPUB spine files
20	<code>fragment_is_title_only</code>	Skips fragments that contain only title text with no body content

2.6 CSS Generation Engine

`css_text()` generates the complete print stylesheet (~400 lines of CSS) driven entirely by the Settings dataclass. Key features:

- **@page rules:** title, front (roman folios), body (Arabic folios), opener (no folios), nofolio (blank pages), :blank (true blank pages)
- **Running head layouts:** Six variants — `right_title_full_rule` (collection title verso, current work recto + full-width vector rule), `centered_single_rule`, `dual_full_rule`, `alternating`, plus two split-rule varieties
- **TOC rendering:** Two-pass system — first pass uses CSS `target-counter(attr(href), page)` with leader dots; second pass uses explicit resolved page numbers with CSS Grid layout for precise alignment
- **Hard normalization:** Aggressive `!important` overrides against ebook CSS (`max-width: none`, `width: auto`) on all body elements
- **Verse styling:** Max-width constraint, hanging indents via negative `text-indent` + `padding-left`, ragged-right alignment, `pre-wrap` white-space
- **Cast list styling:** Constrained max-width, left-aligned
- **Work description styling:** Italic, smaller font (`body_size_pt - 1pt`), no indent, bottom margin
- **Heading hierarchy:** 8 distinct heading levels (major-work, collection-division, backmatter-opener, subdivision, part-heading, chapter-section-heading, h3, minor-heading) each with independent font size, margins, and letter-spacing

2.7 PDF Rendering & Optimization

The rendering pipeline operates in distinct phases:

1. **write_build():** Writes `book.html` and `style.css` to the build directory
2. **render_pdf():** Calls WeasyPrint's `HTML.write_pdf()` with the CSS stylesheet
3. **resolve_toc_page_numbers():** Opens the rendered PDF with PyMuPDF, extracts internal link destinations from TOC pages, maps target IDs to physical page numbers

4. **Second render pass:** If TOC page numbers were resolved, re-renders HTML with explicit page numbers in a CSS Grid layout
5. **subset_pdf():** For sample mode, extracts first N pages using PyMuPDF
6. **draw_vector_runner_rules():** Post-render vector drawing — uses PyMuPDF's Shape API to draw clean stroke-only horizontal lines at the configured `runner_rule_y_mm` position, only on pages that have running-head text
7. **optimize_pdf():** pikepdf compression, object stream generation, and linearization for web-fast viewing

2.8 QA Preflight System

`preflight_pdf()` performs **15+ checks** on the rendered PDF:

Check	Method	Severity
A4 page-size verification	Every page rect measured; flags if width/height deviates >2pt from A4	BLOCKER
Font inventory	PyMuPDF <code>get_page_fonts()</code> across all pages	INFO
Image count	PyMuPDF <code>get_page_images()</code>	INFO
Line-spill detection	Identifies single-letter or Roman-numeral-only lines that look like broken word spills	BLOCKER
Header clearance	Measures Y position of first body text vs configured <code>runner_rule_y + runner_body_clearance</code>	BLOCKER
Narrow column detection	Median text block width < 300pt flagged (A4 body should be ~450pt)	WARN
Blank-page artifacts	Pages with drawings/images but no text	BLOCKER
Empty content pages	After page 4, pages containing only title/folio-like text	BLOCKER
Duplicate title pages	Multiple opening pages matching the book title	BLOCKER
Opener page warnings	Display titles carrying folios; missing blank separators	BLOCKER
TOC duplicate explosion	Entries repeated ≥ 4 times in rendered TOC	BLOCKER
TOC page number uniformity	All entries resolving to the same page number	WARN
First body folio check	Page after TOC should show Arabic folio 1	BLOCKER
Dark-page detection	Rendered PNG pages with average brightness < 45 (via Pillow)	BLOCKER
Work description style	Editorial blurbs must be italic and at the correct smaller font size	BLOCKER

3. AI INTEGRATION

3.1 Providers & Models

The pipeline supports two AI providers via the `--ai-provider` flag:

- **OpenAI** — default provider. Uses `responses.create()` with structured JSON schema for book structure planning and visual QA. Default model: `gpt-5.4-mini`.
- **DeepSeek** — uses `chat.completions.create()` with JSON mode. Default model: `deepseek-chat`. Runs text QA after local deterministic QA completes.
- **none** — disables all AI features; purely deterministic pipeline.

3.2 Structure Planning

`apply_openai_book_plan()` sends spine documents in batches of 24. Each batch includes: document index, href, first 20 extracted headings, text sample (3,500 characters), text length, and local heuristic flags (`contains_poetry`, `contains_drama`, `contains_images`).

The AI returns structured JSON classifying each document as one of: `frontmatter`, `division`, `major_work`, `chapter`, `poetry`, `play`, `backmatter`, `promo`, `local_toc`, or `unknown`. It also returns a `remove_document` flag (only applied at $\geq 86\%$ confidence) and a `major_title` string for work-level heading assignment.

3.3 Image Classification

`ai_image_decision()` sends image src + surrounding context (3,000 characters of caption and nearby text) to the AI for classification as authorial/functional vs. publisher-added plate. The decision is only applied at $\geq 70\%$ confidence. The `should_keep_image()` function implements a multi-tier decision:

1. If `image_policy` is "keep-all" → keep
2. If `image_policy` is "remove-all" → remove
3. If plate context is detected without functional context → remove
4. If AI is available, consult it with confidence threshold
5. If functional context is detected → keep
6. If "cover" or "title" in src with short context → remove

3.4 Visual & Text QA

Visual QA (OpenAI vision)

Renders up to `--openai-qa-pages` (default 10) pages as JPEG images, base64-encodes them, and sends them to the vision model with a 9-category inspection prompt: body justification, paragraph rhythm, chapter/title placement, running heads/rules, folios, TOC, image cleanup, poetry/drama, and ebook artifacts. Returns PASS/FAIL with page-specific findings.

Text QA (DeepSeek or OpenAI)

Sends extracted PDF text (first 28 pages, 60,000 characters) + local QA reports + settings excerpt to the text model for structural review. The model can generate regex rule suggestions stored as `.review.yaml` files using only the 13 known rule-pack keys. These files must be manually reviewed before loading.

4. CONFIGURATION & AUTO-FIX

4.1 Three-Layer Override Precedence

Settings resolve in this exact order:

1. **Built-in defaults** — hardcoded in the `Settings` dataclass
2. **Config file** — YAML or JSON via `--config`; unknown keys are warned and ignored
3. **CLI flags** — ~25 explicit command-line overrides (e.g., `--body-size`, `--line-height`, `--margin-side`)

`write_default_config` generates a complete editable YAML config with all defaults and a comment header explaining the override order.

4.2 Rule Pack System

External YAML files in the `rules/` directory can extend any of the 13 built-in regex patterns. The bundled `generic_epub.yaml` adds:

- 5 promo patterns ("free sample", "about delphi classics", etc.)
- 4 plate caption patterns ("cover image", "title card", "decorative plate", "photogravure")
- 3 functional image clues ("genealogical table", "family tree", "ground plan")
- 3 publisher image source clues ("frontmatter", "titlepage", "halftitle")

Each pattern is validated as a valid regex before merging. AI-generated suggestions are written as `{provider}_rule_suggestions.review.yaml` files.

4.3 Auto-Fix Engine

`auto_fix_settings()` reads QA verdict flags and applies safe CSS/config adjustments in up to `--max-auto-fix-passes` iterations (default 1):

Trigger	Auto-Fix Applied
Header/rule collisions	<code>runner_body_clearance_mm</code> increased by +2mm (max 14mm)
Body typography warnings	<code>justify_prose</code> enabled, <code>hyphenate</code> enabled, <code>line_height</code> increased by +0.025 (max 1.32)
Title placement issues	<code>subdivision_margin_bottom_mm</code> increased by +1.2mm, <code>major_opener_bottom_margin_mm</code> increased by +2mm
TOC issues	<code>toc_line_height</code> increased by +0.02, <code>toc_entry_gap_mm</code> increased by +0.4mm
Folio/numbering issues	Logged for human review; no safe generic auto-fix exists
Image/caption issues	Logged for human review; no safe generic auto-fix exists
Poetry/drama issues	Logged for human review; no safe generic auto-fix exists
Blank/artifact issues	Logged for human review; no safe generic auto-fix exists

After each fix pass, the PDF is fully re-rendered and QA runs again. All CSS auto-fix notes are written to `build_summary.json`.

5. ENHANCEMENT ROADMAP

5.1 Architectural Improvements

Addition	Rationale
Modular package split	Split 3,620-line monolith into <code>pipeline/</code> with ~10 modules (~300–500 lines each). Preserves single-command experience via <code>__main__.py</code> while gaining testability, maintainability, and contributor accessibility.
Pipeline as DAG	Model each transformation as a directed acyclic graph node. Enables: (a) caching intermediate results, (b) parallel document processing, (c) selective re-execution when only one stage changes.
Caching layer	Save cleaned HTML per document keyed by content hash. When tweaking <code>--line-height</code> or margins, the entire scan/classify/clean phase is skipped — only CSS + render reruns. Saves minutes per iteration on large collected works.
Plugin system	Beyond regex rule packs: register custom Python cleaners, classifiers, and post-processors via a plugin hook system without modifying core code.

5.2 Missing Typographic Features

Feature	How It Fits the "Deluxe" Vision
Drop caps / initial caps	Detect first paragraph after chapter headings; wrap first letter in <code></code> with CSS <code>initial-letter: 3</code> . A defining feature of deluxe print editions.
Small caps normalization	Distinguish ALL-CAPS acronyms from semantic small caps (AM/PM, BC/AD, abbreviations). Convert to <code>font-variant: small-caps</code> . Critical for scholarly typography.
Ligature control	Add configurable <code>font-variant-ligatures</code> with per-element overrides. EB Garamond has rich ligature sets that professional typography controls deliberately.
True footnote placement	Collect footnote bodies, place at page bottom with rule separator using WeasyPrint <code>float: footnote</code> . Currently footnote bodies are lost or become inline text.
Post-render widow/orphan check	CSS <code>widows:2; orphans:2</code> is advisory — real rendering often produces widows anyway. A post-render analysis catches and flags actual widow/orphan occurrences by page.
River detection	Image-processing on rendered PNG pages: detect vertical white-space gaps in justified text via connected-component analysis. Rivers are a hallmark of poor justification and are explicitly mentioned in the production prompt.
Page balance check	Detect chapters where the last page carries fewer than 4–5 lines. Professional typesetters avoid "widow pages" with only 1–2 lines.
Table intelligence	Detect wide tables, optionally rotate to landscape on their own page. Auto-fit column widths. Handle multi-page tables with repeated headers. Essential for genealogical tables and chronologies.
Margin notes / sidenotes	Detect note-like short paragraphs and convert to CSS <code>@left-middle</code> or float-based sidenotes. Common in annotated classics.
Multi-column layouts	Support 2-column mode for indices, bibliographies, and chronologies. Configurable per-document-kind.

5.3 Print-Production Additions

Feature	Description
Crop marks + bleed	For offset printing: 3mm bleed with trim marks. Add <code>--bleed-mm</code> and <code>--crop-marks</code> . WeasyPrint can render larger pages with the live area centered; trim marks as CSS borders or vector paths.
ICC color profile embedding	For CMYK prepress: embed FOGRA39 or similar ICC profile into PDF. pikepdf can inject OutputIntent. Add <code>--color-profile</code> flag.
Spine width calculator	Given page count + paper GSM, calculate spine width in mm. Output to <code>build_summary.json</code> to feed directly into cover design workflow.
Signature imposition hints	Output page count organized into printer signatures (typically 16 or 32 pages). Include in build summary for binding layout.
PDF/X compliance	Optionally output PDF/X-1a or PDF/X-4 for guaranteed prepress compatibility. pikepdf can validate and add required metadata.

5.4 QA & Safety Enhancements

Feature	How It Works
Interactive QA accept/reject	<code>--interactive-qa</code> mode: display each QA warning, let user press <code>a</code> (accept/suppress), <code>f</code> (flag for fix), <code>i</code> (ignore). Stores decisions in a state file for future runs.
Diff-against-reference	<code>--diff-against golden/</code> compares QA reports and flags only <i>new</i> issues. Essential for iterative refinement of a known-good build.
Visual diff renders	Side-by-side rendered PNGs of the same pages from two builds, with pixel-diff overlay highlighting changed areas in red.
Regression test corpus	A <code>tests/</code> directory with small curated EPUBs exercising specific features. CI/CD runs <code>--strict</code> on all and checks exit codes.
Rule pack validator	Before using a new rule pack, run it against a corpus: "This pattern would remove X paragraphs from Y documents, including sample text Z." Prevents accidentally deleting real literature.

5.5 Workflow & UX

Feature	Description
Incremental/ resume builds	If a build crashes after processing 200/500 documents, save progress and resume. Valuable for huge collected works (10+ minute processing).
Batch processing	<code>--batch "books/*.epub" --config shared.yaml</code> processes multiple EPUBs sequentially with shared settings.
Watch mode	<code>--watch</code> monitors EPUB and config. On change, auto-rebuilds the sample and opens the PDF. Tight feedback loop during style tuning.
Progress reporting	Rich progress bars (tqdm): scanning spine, cleaning documents, rendering PDF, running QA — with estimated time remaining.
Config validation	Validate Settings before building: positive margins, font size in valid range, font files exist, runner rule Y position compatible with clearance. Catch misconfigurations early.
Template-based CSS	Replace string-concatenation <code>css_text()</code> with Jinja2 templates. Separates layout logic from Python code; makes custom CSS variants trivial.

5.6 Output Diversification

Format	Use Case
Cleaned EPUB output	After cleanup, also produce a cleaned EPUB. Same pipeline, dual output. Useful for e-reader distribution alongside the print PDF.
Static HTML website	Generate a static HTML version with navigation. Single-source multi-format publishing from one pipeline.
PDF variants	<code>--profile digital</code> (RGB, optimized), <code>--profile print</code> (CMYK, bleed, crop), <code>--profile proof</code> (watermarked, low-res "PROOF" overlay).

5.7 Non-Latin & Accessibility

Feature	Description
Non-Latin script support	All regex patterns assume Latin script. Add CJK, Cyrillic, Arabic, Devanagari detection modes. Poetry detection uses Latin-specific heuristics that fail for CJK.
RTL language support	Right-to-left page progression, RTL running heads, RTL TOC. WeasyPrint supports <code>direction: rtl</code> — just needs a pipeline flag.
PDF/UA accessibility tagging	Add document structure tags for screen-reader-compatible PDFs. pikepdf can inject structure trees.
Metadata passthrough	Currently only title is extracted. Also pull: author, publisher, date, language, ISBN. Embed in PDF metadata and optionally on a colophon page.

6. CONCLUSION & PRIORITY MATRIX

The `deluxe_epub_to_pdf.py` pipeline is already a remarkably capable and well-engineered tool. It handles the hardest problems in EPUB-to-print conversion — structural classification of malformed public-domain EPUBs, promotional content removal, image triage, poetry/drama normalization, and rigorous QA — with a level of thoroughness that surpasses most commercial and open-source alternatives.

The following matrix organizes the proposed enhancements by impact and effort:

Priority	Enhancement	Impact	Effort
HIGH	Modular package split	Foundational — enables all other work	Medium
HIGH	Caching layer	Massive iteration speed improvement	Medium
HIGH	Config validation	Prevents silent misconfigurations	Low
HIGH	Drop caps + small caps	Defining "deluxe" typographic feature	Medium
HIGH	True footnote placement	Scholarly edition requirement	High
MED	Template-based CSS (Jinja2)	Cleaner separation, easier customization	Medium
MED	Widow/orphan + river detection	Professional QA upgrade	Medium
MED	Crop marks + bleed + ICC profiles	Professional print-shop readiness	Medium
MED	Progress reporting (tqdm)	UX improvement for long builds	Low
MED	Watch mode	Tight feedback during style tuning	Medium
LOW	Plugin system	Extensibility for power users	High
LOW	Clean EPUB / HTML output	Multi-format publishing	Medium
LOW	Non-Latin / RTL / PDF/UA	Broader audience, accessibility	High
LOW	Interactive QA / visual diff	Power-user QA workflow	High
LOW	Batch processing	Multi-book workflows	Low

Recommended First Steps:

1. Modular package split — the single-script monolith is the biggest barrier to all further development.
2. Caching layer — makes style-tuning iteration nearly instant for large books.
3. Config validation — low effort, high safety payoff.
4. Drop caps + small caps — these are the two most visibly "deluxe" features currently missing from the typographic output.

Report generated from the project: `local_deluxe_epub_pipeline_configurable_with_guide`

Source script: `deluxe_epub_to_pdf.py` (~3,620 lines) · Date: June 2026